

LapSim engineering handoff

Mathematics, Python architecture, desktop interface, data provenance, verification, and known limits

Scope: current source-launched desktop application, its two simulation modes, their input data, and the code and checks needed to maintain them.

Audience: software lead and vehicle engineers.

Status: implementation description, not vehicle certification or a claim of predictive accuracy.

Source basis: this repository; the supplied *EV Vehicle Simulation and Torque Control* research report (Report I); and the supplied *EV Vehicle Simulation II: Aerodynamics, CFD, Track Conditions, and Secondary Effects* (Report II, 5 October 2026). Report II's proposals are treated as design evidence, not as proof that a feature exists or that its example values describe the team car.

Source of truth: docs/engineering_handoff.md

Contents

1. Read this first	4
1.1 Evidence levels	4
2. Run the code and locate the outputs	4
3. Repository architecture and execution boundaries	5
3.1 State ownership and side effects	5
4. Frames, signs, units, and terms	6
5. Data path and provenance	6
5.1 Course input	6
5.2 Car profiles and actual model values	7
5.3 Missing physical data	7
6. Main lap mode: aero, suspension, and tire capacity	8
6.1 Aerodynamic forces	8
6.2 Quasi-static suspension loads	8
6.3 Tire law and combined force	9
7. Main lap mode: powertrain, brakes, and battery	9
7.1 Propulsion and equivalent rotating mass	9
7.2 Hydraulic braking and regenerative allocation	9
7.3 RC battery and energy sign	10
8. Main lap mode: speed ceiling, controller, and cell update	10
8.1 Precompute the path speed envelope	10
8.2 Driver/controller action	10
8.3 Advance the vehicle by one distance cell	11

8.4 Optional experimental racing-line planning	11
9. What the main GUI displays	13
9.1 Driver playback data flow	14
10. Four-wheel lab: state, control, tire, and motion equations	14
10.1 Power identity and integration	15
11. Report II upgrade: wind and road conditions	15
11.1 What Report II does and does not authorize	16
12. Four-wheel GUI inputs and outputs	16
13. Profile, source-data, and model audit trail	17
14. Saved records: exact schema, integrity, and replay limits	17
14.1 Main-lap record (`experiments/run_record.py`)	17
14.2 Four-wheel A/B record (`experiments/dynamics_record.py`)	18
14.3 Practical record inspection	19
15. How Python modules and the desktop interact	19
15.1 What to change for a new car profile	20
15.2 What to change for a new track	20
15.3 Numerical and code-review guardrails	20
16. Checks performed, interpretation, and known discrepancies	20
17. Prioritized engineering work and acceptance evidence	22

1. Read this first

LapSim has **two different calculation paths**. The main desktop tab estimates a flying lap along a prescribed centerline. It advances from one distance cell to the next, using the repository's vehicle, powertrain, battery, brake, aero, tire, and quasi-static suspension models. Its output is a model estimate for the selected car and course. The separate four-wheel lab integrates world position, body velocity, yaw, and four wheel speeds over time for a short, synthetic, open-loop torque-allocation maneuver. It is not an endurance-lap solver and does not use the selected Prius or TREV car profile.

That distinction is the most important interface contract in the project. A lap-time difference is not proof that the time-domain torque controller will be faster; a yaw difference in the lab is not a lap-time prediction. Neither mode currently has a calibrated 2026-27 car. The built-in Prius uses an equivalent electric power source to exercise the software, not a Toyota hybrid transaxle model. Optional TREV working profiles use partial local evidence and retain inherited defaults for missing subsystems.

The work informed by Report II adds explicit air-relative wind and per-wheel road-grip conditions to the four-wheel experiment. The **core API's** zero-drag, still-air, base-grip default recovers the pre-extension model; the desktop deliberately starts with a synthetic nonzero drag-area input. Full six-component CFD maps, ride-height feedback, vertical road/contact dynamics, and temperature-dependent limits remain future work because the required car, tire, CFD, road, and thermal data are not in this repository. Section 17 lists the exact data needed to advance those models.

An optional **experimental racing-line planner** now proposes one smooth path within an explicitly supplied corridor and times it against a consistently processed geometric centerline using the main lap model. The ordinary source-curvature centerline remains the default and does not run optimization. The planner is deterministic geometry optimization rather than machine learning or closed-loop driving. The driver view plays solved telemetry on a reference map; it is not a measured or simulated tracked vehicle pose. Section 8.4 gives its math, Section 9 describes the desktop presentation, and `docs/ai_racer_design.md` records its research and evidence limits.

1.1 Evidence levels

Level	What it establishes here	What it does not establish
Equation and unit checks	Signs, conservation identities, limiting cases, numerical behavior of implemented equations	Parameters of the real car
Synthetic scenario check	Software responds to controlled changes in torque, wind, or grip	A measured lap-time or controller gain
Historical replay or fitted parameter	Agreement or disagreement with the recorded conditions and channels used	Independent prediction when parameters were tuned on the same run
Team-car validation	Would require held-out, synchronized measurements with configuration and uncertainty	Not currently available for the complete 2026-27 simulator

The older report's 44 reference checks belong to its separate source package. They were **not** tests of this repository. The repository's current checks are identified in Section 16.

2. Run the code and locate the outputs

The supported desktop launch is from the repository checkout on Windows:

```
cd C:\Users\lucas\Documents\Codex\LapTimeSim
.venv\Scripts\python.exe -m pip install -e .
.venv\Scripts\python.exe -m lapsim.ui
```

`launch_lapsim.cmd` uses the same local virtual environment. The VS Code launch configuration is in `.vscode/launch.json`. The application is native Tkinter with embedded Matplotlib; there is no distributable EXE. Its course loader currently expects the checkout-relative `analysis/data/track/gnss_imu_endurance_track.csv`. Packaging the Python module alone does not

package that course.

Run the repository checks with:

```
$env:OPENBLAS_NUM_THREADS='1'  
.venv\Scripts\python.exe -m pytest -q
```

The first line is a Windows runtime setting used during project checks to avoid unnecessary numerical-library threads. It is not a physics parameter. A passing suite establishes the stated software checks, not full vehicle accuracy.

This Markdown file is the editable report source. With ReportLab installed in a documentation Python environment, `python docs/render_engineering_handoff_pdf.py` regenerates `output/pdf/LapSim_Engineering_Handoff.pdf`; ReportLab is not required by the simulator itself.

Local user profiles are saved in `%LOCALAPPDATA%\LapSim\car_profiles.json`. Completed and interrupted main-lap runs are saved in `%LOCALAPPDATA%\LapSim\runs`; four-wheel A/B runs are saved in `%LOCALAPPDATA%\LapSim\dynamics_runs`. These files are outside Git; do not confuse them with the built-in or reviewed-source profiles. The status bar shows a shortened record ID; the file name contains the complete ID. Record contents are described in Section 14.

3. Repository architecture and execution boundaries

```
src/lapsim/ui/__main__.py  
-> ui/app.py: Tk controls, worker thread, plots, profile comparison,  
    optional racing-line workflow, driver playback, WIP tab  
-> ui/garage.py and profiles/adapters.py: car construction and provenance  
-> ui/simulation.py: course load/resample and default run_one_lap  
-> optimization/racing_line.py: optional processed x/y baseline,  
    spline candidate, then run_one_lap twice  
-> solvers/path_constraints.py: cyclic speed ceilings  
-> events/endurance.py: driver requests, braking, cell advance  
    -> vehicle_model/vehicle.py: component coordination  
        -> tire, suspension, aero, brake, drivetrain, battery  
-> experiments/run_record.py: settings, results, aligned telemetry  
-> ui/driver_view.py: time/distance interpolation onto reference x/y  
  
-> ui/dynamics_lab.py: separate short maneuver and A/B displays  
-> dynamics/planar.py: time-domain four-wheel forces and RK4  
-> dynamics/conditions.py: deterministic wind and road queries
```

The environment is a pure model queried during integration, not a GUI animation. `vehicle_model/interfaces.py` defines replaceable component contracts. `lapsim` owns courses, solvers, controls, events, optimization, telemetry, profiles, and experiment records. The `analysis/` tree contains source data, historical studies, preparation scripts, and archived material; its scripts do not run every time the desktop app starts.

`events/api.py::simulate_endurance` is a separate generic event/scoring interface. `solvers/speed_limit.py` and `solvers/lap_time.py` are other solver utilities. The desktop's actual one-lap route is `ui/simulation.py::run_one_lap` through `PathConstraintSolver` and `EnduranceSimulator`. Do not use a result from one API as if it documented the assumptions of another.

3.1 State ownership and side effects

The main `Vehicle` is mutable. `run_one_lap` resets it before a run. Component `update_state` methods commit accepted cell results; telemetry is recorded after accepted cells. The time-domain lab uses immutable state and control values for force evaluations. Its Runge-Kutta stages may call a force function repeatedly, so road and wind queries must be pure: an internal substep must not draw a new random weather value, heat a tire, drain a battery, or change a controller command. Report II Section 12 explicitly distinguishes force evaluation from committed state evolution.

The GUI is not the solver's clock. Lap work runs on a worker thread and returns results through a Tk queue. The four-wheel lab also computes away from the Tk event loop. Redrawing a plot or moving the time cursor must not change a run's physics

or data.

4. Frames, signs, units, and terms

Symbol or channel	Meaning	Positive direction / units
s, distance_m	Distance along prescribed lap path	Increasing, m
X,Y	World-plane position in the four-wheel lab	World x/y, m
Body x,y,z	Vehicle axes	Forward, left, up
psi, r	Heading and yaw rate	Counterclockwise about +z, rad and rad/s
u,v	Body center-of-gravity velocity	Forward and left, m/s
kappa	Path curvature or wheel longitudinal slip, as named by context	Path: 1/m; tire slip: dimensionless
Fx,Fy,Fz	Tire force	Forward, left, upward reaction, N
omega	Wheel angular speed	Forward rolling positive, rad/s
Tdrive,Tbrake	Signed wheel drive and applied friction-brake torques	N m; brake torque opposes spin
Ppack	Battery terminal power in lap mode	Positive discharge, negative charge, W
rho	Air density	kg/m ³
w	Air velocity in world frame	m/s; distinct from car ground velocity

The four-wheel wheel order is **front-left, front-right, rear-left, rear-right**. Wheel body offsets use +y for the left side. A stronger right-rear forward tire force creates positive yaw because $M_z = x F_y - y F_x$ and the right wheel has negative y. The lap solver uses a prescribed scalar path curvature; it does not integrate yaw dynamics with those independent wheel forces.

All public physics inputs are SI except fields that explicitly say rpm, psi, kWh, deg, or km/h. A dimensionless tire friction coefficient is not a force. The visible GUI converts speed from m/s to km/h, acceleration to g with 9.80665 m/s², forces to kN in traces, and battery power to kW. Those are display conversions; the core models use SI. The report's equations below use ASCII variable names so the exact code convention is clear.

5. Data path and provenance

5.1 Course input

analysis/data/track/gnss_imu_endurance_track.csv is the desktop's shipped 989 m course. The accompanying JSON says it is a spatial fusion of GNSS position, corrected IMU lateral acceleration, and an earlier schematic/registered centerline; it is not a surveyed current Formula SAE course. The metadata records 0.5 m source spacing, 1,979 points, a 0.76 m closure error, and limitations on map alignment and low-speed curvature. Some historical absolute file paths in that JSON describe the original preparation workstation; they are provenance strings, not valid paths on this computer.

SpatialTrack reads point arrays distance_m, x_m, y_m and one curvature value per cell. It checks finite values, channel lengths, and strictly increasing distance. It does **not** prove that the plotted x/y geometry exactly matches the cell curvature, that a closed flag closes the coordinates perfectly, or that the car remains within cones. The path solver consumes distance, cell length, and curvature. The plotted map consumes x/y. Vehicle.update_state separately integrates its achieved x/y/heading; the desktop does not overlay this path or check track width.

This difference is material to the optional planner. `SpatialTrack.geometry_audit()` now exposes a repeatable check: the shipped CSV has **989 m** of source station distance versus about **1,012.35 m** of accumulated x/y chords, about **0.760 m** endpoint gap, and **1,441 of 1,978 individual x/y chords longer than their assigned station intervals**, totaling 45.23 m of positive excess. A path segment cannot have a chord longer than its own travel distance. A separate heading audit found about **6.09 rad** total x/y heading turn versus about **3.66 rad** integral of the saved curvature. The Analysis tab displays a warning; the data channels are separately fused estimates rather than one self-consistent path. The experimental mode therefore generates both its baseline and proposed line from the same prepared x/y geometry and recomputes curvature for both. Its time comparison is *not* relative to the ordinary 989 m source-curvature lap. The source metadata provides no surveyed left/right boundary, so a width entered in the desktop is a scenario assumption.

`ui/simulation.py::resample_track` creates cells at the requested maximum length. For source cells overlapping a new cell `[a,b]`, it computes

```
kappa_new = sum(kappa_old_i * overlap_length_i) / (b - a).
```

This conserves the integrated signed curvature over each new cell. It linearly interpolates x/y at the new distance boundaries. Coarsening still removes short curvature variation and can change lap time. The default GUI step is 1 m; 0.5 m uses the source's nominal station spacing. Solver step is a physical/numerical modeling choice, not a plot-resolution option.

5.2 Car profiles and actual model values

`profiles/adaptor.py::build_vehicle` constructs a **fresh** vehicle and a manifest. `repository_baseline` uses repository defaults. `prius_2026_1e` uses `ui/presets.py::make_prius_benchmark` and an idealized one-motor, front-drive power envelope based on published combined power. The Prius's mass and wheelbase are published benchmark specifications; its effective motor curve, pack, grip, and several aero/chassis inputs are assumptions. The saved-user profile mechanism (`ui/garage.py`) copies only seven editable Prius-equivalent fields: mass, peak-equivalent power, wheelbase, tire radius, tire friction, drag area, and speed limit. It is not a general editor for every Python model parameter.

The built-in Prius starter values are 1404.78 kg converted from 3097 lb, 144.67 kW converted from 194 hp combined power, 2.7508 m wheelbase converted from 108.3 in, and 0.3329 m **nominal geometric** radius from tire designation 195/60R17. Those four conversions are drawn from the manufacturer-specification block in `ui/presets.py`; geometric radius is not a measured loaded rolling radius. The editable starting assumptions are tire $\mu=0.95$, $C_dA=0.59 \text{ m}^2$, and top-speed cap 180 km/h. The constructed equivalent has a front driven axle, a four-knot idealized motor curve with a 4200 rpm knee and 10000 rpm maximum, unit motor/inverter/chain efficiencies, no charge-power allowance, zero aero lift, 60% static front weight, 0.53 m CG height, 1.56/1.58 m front/rear tracks, rolling-resistance coefficient 0.012, and cornering-drag coefficient 0.025. These are **software-model assumptions**, not a claim to reproduce a Toyota hybrid's mechanical or electrical architecture. Exact constructed values and overrides are frozen in each lap record.

Optional TREV5 working profiles require the reviewed ENME408 data package in Downloads. `profiles/registry.py` validates the package and `profiles/adaptor.py` promotes only allowlisted, unit-compatible fields. The manifest separates selected source values, inherited model defaults, unused records, conversion details, source locations, artifact hashes, and limitations. A source browser can inspect records without silently applying them. These profiles are partial working scenarios; neither motor count nor 2026-27 electrical topology should be inferred from a label.

The effective vehicle snapshot in each saved lap record is the **constructed car used by that run**, not merely the chosen base profile. Editable overrides are recorded separately. This matters because a user can select Prius, change a number, run once, and leave the named profile unchanged.

5.3 Missing physical data

The shipped course has no track width, elevation, bank, crest radius, local surface material, wetness, roughness, road temperature, or measured wind. The main lap mode therefore assumes a flat, uniform, still-air path with its configured constant density. A road patch drawn in the four-wheel lab is a deterministic synthetic sensitivity input, not a reconstructed patch from this course. No rain graphic is treated as tire physics.

6. Main lap mode: aero, suspension, and tire capacity

The lap model is quasi-static within each spatial cell. It does not solve a Navier-Stokes fluid field, suspension differential equations, or four independent wheel angular speeds. Its components provide algebraic forces to the path solver and the vehicle's cell update.

6.1 Aerodynamic forces

`vehicle_model/aero/model.py::Aero.forces_n` computes dynamic pressure $q = 0.5 \rho v^2$, positive drag $D = q C_d A$, and signed downforce $D_{\text{down}} = -q C_l A f_{\text{roll}}$. $C_l < 0$ means downforce under this model's convention. The front share is p_{Ddown} ; the rear gets $(1-p) D_{\text{down}}$. The roll factor is

```
f_roll = 1 - min(abs(roll)/roll_limit, 1) * (1 - retention_at_limit).
```

At and above the default one-degree roll limit, the default retains 50% of the zero-roll downforce. This is a configured approximation, not a measured ride-height/attitude map. `Vehicle.aero_forces_n` obtains a quasi-static body-roll estimate from suspension before calling `Aero.forces_n`. The optional `ActiveAero` subclass selects a two-position low-drag state from absolute path curvature and scales drag and downforce independently; it has no actuator delay or rule-eligibility determination. The desktop's default Prius uses its own estimated $C_d A$, not the baseline aero package's coefficient table.

This lap calculation currently uses **ground speed as airspeed** and a fixed air density. It does not include crosswind side force, aero yaw/pitch moments, or the wind-aware model in the separate lab. Its baseline coefficient and reference area values are listed in `docs/model_parameters.md`. The active ratio $\text{abs}(C_l)/C_d$ must be computed from those actual coefficients; a historical unused ratio constant was removed because it disagreed with them.

The total normal reaction used by resistance and tire calculations is approximately $m g + D_{\text{down}}$ on the flat path. A positive-lift configuration that removes all reaction or makes either axle's base reaction negative is now rejected as outside this constrained-contact model. It is not silently turned into extra tire grip or a negative denominator. The code still cannot simulate flight, porpoising, or a crest.

6.2 Quasi-static suspension loads

`vehicle_model/mech/suspension.py::tire_normal_loads_n` starts from static axle weights and adds the configured front/rear aero downforce. Longitudinal load transfer is proportional to $m a_x h_{\text{CG}} / L$, moving load rearward under forward acceleration. Lateral transfer uses $F_y = m a_y$, front/rear roll-axis heights, track widths, total elastic roll stiffness, and the configured front load-transfer distribution. It estimates body roll for the aero multiplier. Wheel loads are clipped at wheel lift only within its algebraic model; it has no heave, pitch, roll, damper, unsprung-mass, or contact-trajectory states. Do not add a second independent load-transfer correction around this component.

More specifically, its quasi-static roll angle is $\phi = m a_y \max(h_{\text{CG}} - h_{\text{roll_axis_at_CG}}, 0) / K_{\text{roll}}$. Geometric roll moments act through front/rear roll-axis heights; elastic roll moment is distributed according to a front stiffness fraction chosen to approximate the configured front lateral-load-transfer fraction within $[0, 1]$. Each axle's lateral transfer is its geometric plus elastic moment divided by that axle's track, then limited so no algebraic wheel load is negative. The exact stiffness-fraction solve is in `suspension.py::tire_normal_loads_n` and is included in the effective vehicle snapshot; it is an equilibrium allocation, not a transient suspension response.

Report II gives a more complete flat-road pitch balance including aerodynamic pitch moment and longitudinal ground force. The present model is **not** an exact solution of that general balance: it has no aerodynamic pitch moment, wind, grade, or transient pitch angular momentum. Its outputs are suitable for model sensitivity and force-budget checks under the stated flat, quasi-static assumptions.

6.3 Tire law and combined force

`vehicle_model/mech/tire.py::Tire` is the default main-lap tire. It interpolates a load-sensitive $\mu(F_z)$ table over its configured range and clamps beyond the table. Pure lateral and longitudinal capacities are $\mu_{lat}(F_z) \max(F_z, 0)$ and $\mu_{long}(F_z) \max(F_z, 0)$. It distributes a requested total lateral force across wheels in proportion to capacity. Its remaining longitudinal limit under lateral force is a friction-circle-style capacity:

$$F_{x_remaining} = F_{x_pure} * \sqrt{\max(0, 1 - (F_y / F_{y_pure})^2)}.$$

Drive demand is allocated only to configured driven wheels; braking requests are distributed by axle and constrained at each contact patch. The default main-lap tire does not integrate each wheel's $J \, d(\omega)/dt$. It infers a longitudinal slip estimate from force/capacity, with a configured peak-slip ratio and optional distance relaxation. Tire saturation and the path speed ceiling are therefore not a substitute for a measured force-versus-slip-and-temperature map.

Optional Pacejka classes and a public UPC `.tir` file exist in `vehicle_model/mech/pacejka.py` and `data/tires`. They are **not** silently enabled by the desktop. The imported file's pure lateral information and its longitudinal/combined-slip provenance differ; see `docs/model_parameters.md` before using them as a measured complete tire model.

7. Main lap mode: powertrain, brakes, and battery

7.1 Propulsion and equivalent rotating mass

The baseline power chain is DC pack terminal $\rightarrow$ inverter $\rightarrow$ motor shaft $\rightarrow$ chain/final drive $\rightarrow$ driven tires. `motor.py` interpolates peak torque against RPM and limits shaft power and maximum RPM. `drivetrain.py` uses a nominal no-slip conversion from ground speed through the tire rolling radius and gear ratio for its envelope; cell force/power checks can then use inferred driven-wheel surface speed from tire slip:

$$\begin{aligned} \omega_{\text{wheel}} &= v / R; \omega_{\text{motor}} = \text{gear_ratio} * \omega_{\text{wheel}}; \\ F_{x_wheel} &= T_{\text{motor}} * \text{gear_ratio} * \eta_{\text{chain}} / R. \end{aligned}$$

The available motor torque also depends on pack discharge power, inverter and motor efficiencies, and motor power limit. `vehicle_model/vehicle.py` can further reduce demand when the driven wheel surface speed would violate the motor envelope or configured vehicle speed cap. The default efficiencies are constants, so there is no torque-versus-temperature derating or detailed inverter loss map. The continuous motor curves are available as `references/telemetry`; the one-lap event does not thermally integrate continuous rating usage.

Main-lap wheel, shaft, and rotor inertias are **reflected into an equivalent longitudinal mass** $m_{\text{eff}} = m + J_{\text{ref}}/R^2$, where $J_{\text{ref}} = (J_{\text{motor_rotor}} + J_{\text{chain_input}}) \text{ratio}^2 + J_{\text{chain_output}} + J_{\text{driven_wheels}}$. This belongs only to the distance-domain car, which does not integrate wheel speed as independent state. The four-wheel time-domain model instead integrates four $J \, d(\omega)/dt$ equations and must not also add the same wheel inertia to m . This separation prevents double counting.

The built-in repository defaults include parameters that were fit to the historical endurance run: rolling radius, motor efficiency, and chain efficiency. Agreement with that same run is in-sample consistency. It cannot by itself validate a prediction for a future car, another tire, or another track.

7.2 Hydraulic braking and regenerative allocation

`vehicle_model/mech/brakes.py` converts independent front/rear pressure requests, after its configured deadband/map, to axle torque and then force through rolling radius; both pressures have a 300 psi baseline ceiling. Tire force is the final cap. The endurance automatic path controller asks for enough negative force to reach the next speed ceiling and distributes friction brake requests across the two axles and their left/right tires. The pressure inverse is an actuator request, not a guarantee that the path is feasible.

`Vehicle.maximum_regenerative_brake_force_n` limits a requested regen magnitude by motor speed/torque, drivetrain efficiency, and pack charge power. `Vehicle.update_state` checks driven-axle topology, and `Tire.calculate_forces` then limits

the combined regen and friction-brake request by contact grip. EnduranceSimulator enables automatic regen only if a non-null SOC threshold is configured and SOC is below that threshold. The desktop EnduranceRunConfig does **not** set this threshold, and the baseline/Prius default pack charge-power cap is zero. Thus a visible regenerative-braking trace may correctly stay at zero. Friction-brake heat is not modeled, and regenerative energy is signed battery power rather than disc heating.

7.3 RC battery and energy sign

The repository default is a one-RC Thevenin pack in `vehicle_model/electrical/battery.py`; the Prius benchmark creates an `OCVPackBattery` equivalent. Both use pack open-circuit voltage versus SOC and a terminal resistance. For the RC variant, with positive current discharging:

```
V_source = OCV(SOC) - V_polarization;
V_terminal = V_source - I R0;
P_terminal = V_terminal I;
I = 2 P_terminal / (V_source + sqrt(V_source^2 - 4 R0 P_terminal));
SOC_next = clip(SOC - I dt / (3600 capacity_Ah), 0, 1);
V_polarization,next = exp(-dt/(R1 C1)) V_polarization
                    + R1 (1 - exp(-dt/(R1 C1))) I.
```

The low-current quadratic root avoids cancellation near zero power. Pack limits also bound terminal voltage, discharge/charge power, SOC, and the quadratic branch. A positive `battery.power_w` means discharge; negative means charging. The event's displayed **net energy** integrates signed terminal power over accepted cells and converts joules to kWh. The telemetry recorder also has a separate cumulative **positive-discharge** energy channel; these must not be interchanged. The pack has no cell temperature, cooling, aging, or measured Prius hybrid energy flow.

`docs/battery_model.md` and `docs/battery_rc_validation.md` describe the model and the limited historical voltage holdout.

8. Main lap mode: speed ceiling, controller, and cell update

8.1 Precompute the path speed envelope

`ui/simulation.py` builds a constant-fraction, two-knot periodic torque profile; both knots have the same requested fraction, so this desktop input is a constant normalized driver demand. It creates `PathConstraintSolver` with explicit tolerance/pass/iteration settings from `path_solver_settings`. At each curvature cell the solver bisection speed between zero and the configured maximum until required lateral force $m v^2 \text{abs}(\kappa)$ fits the sum of quasi-static tire lateral capacities. The local limit is then passed through cyclic backward braking passes, using the next cell's speed and available braking force to bound each entry speed. Conceptually:

$$v_{\text{entry}}^2 + 2 a_{\text{brake}}(v_{\text{entry}}, \kappa, \text{car}) \Delta s \leq v_{\text{next}}^2.$$

The implemented braking acceleration is nonlinear in speed and load transfer, so the solver evaluates force balance and bisection rather than assuming one constant deceleration for the entire lap. It iterates around the closed path until its configured convergence tolerance or pass bound is reached. `PathSpeedConstraints` stores local corner limits and braking ceilings for every cell. A nonzero speed ceiling is a model output, not a safety guarantee for a real car.

8.2 Driver/controller action

`events/endurance.py::EnduranceSimulator.run` resets the vehicle and, unless an explicit start speed was supplied, begins at the **first cyclic braking ceiling**. The desktop therefore reports a **flying lap**, not a standing-start acceleration event. Before each cell it checks entry against local and braking ceilings. The automatic torque-profile controller converts driver fraction to requested motor torque, computes the force needed to reach the next ceiling, and chooses one of drive, coast/torque reduction, or hydraulic braking (with optional configured regen). Steering is chosen from the prescribed curvature with $\delta = \text{atan}(\kappa L)$. A full `EnduranceControlProfile` can instead supply explicit torque, pressure, regen, and steering; that path is checked against constraints rather than silently repaired.

The automatic brake allocator inverts an estimated axle-force request into pressures. When either axle reaches the effective configured pressure cap, the controller now commands both axles at that cap, matching the full-pressure premise of the

backward braking envelope. The committed vehicle load-transfer solve can otherwise deliver slightly less force than the inverse estimate; small deficits accumulated into an apparent ceiling failure in one TREV5 working scenario. The tire combined-slip model still caps contact-patch force, and pressure never exceeds the configured or physical limit. This is a conservative simulation guard, not a calibrated brake-bias or ABS controller; a prospective pressure solve against the actual vehicle step remains future work.

If the vehicle exceeds a ceiling, stops, exceeds time, depletes SOC, or cannot traverse a cell, `EnduranceRunResult` carries a failure reason and completed-lap count. A failed comparison is not ranked as a valid lap. The desktop saves its record for diagnosis. One bookkeeping detail matters: `Vehicle.update_state` commits the cell before the post-step next-ceiling check; if that check fails, the result time/state can include the rejected cell while telemetry is recorded only for accepted cells. Do not assume the last telemetry timestamp equals the result time for a failed run.

8.3 Advance the vehicle by one distance cell

`Vehicle.update_state(controls, delta_s)` obtains tire, brake, motor, aero, and suspension forces and solves a longitudinal force-balance fixed point. Required lateral force is $m v_{\text{entry}}^2 \text{abs}(\kappa_{\text{request}})$. If tire capacity is lower, a root solve reduces achieved lateral force and therefore effective curvature. Resistance is

$$\begin{aligned} F_{\text{resist}} &= D_{\text{aero}} + C_{\text{rr}} (m g + D_{\text{down}}) \\ &\quad + C_{\text{corner}} F_y^2 / (m g + D_{\text{down}}). \\ a_x &= (F_{x_{\text{tire}}} - F_{\text{resist}}) / m_{\text{eff}}. \end{aligned}$$

The normal-load guard rejects unsupported negative reaction before the denominator is used. The cell kinematics assume constant longitudinal acceleration at the cell entry operating point:

$$\begin{aligned} v_{\text{exit}}^2 &= v_{\text{entry}}^2 + 2 a_x \text{delta}_s; \\ dt &= 2 \text{delta}_s / (v_{\text{entry}} + v_{\text{exit}}). \end{aligned}$$

Time and force are coupled because optional tire slip relaxation may depend on dt . The code iterates to a relative time tolerance and refuses a cell it cannot traverse. It then commits component states using the converged dt , integrates effective heading by $\text{delta_heading} = \kappa_{\text{effective}} \text{delta}_s$, and adds the corresponding circular-arc x/y increment. This is a spatial, quasi-static approximation. It does not integrate lateral velocity, yaw inertia, road banking, or track containment.

8.4 Optional experimental racing-line planning

`src/lapsim/optimization/racing_line.py` prepares a **new path input** for the unchanged lap model. It does not change the force, power, braking, or battery equations in Sections 6-8. The desktop imports this module only after **AI racing line (experimental)** is selected. Ordinary **Centerline (default)** still calls `resample_track` and `run_one_lap` on the shipped distance/curvature channel and pays no optimization cost. The AI route requires the user to enter assumed half-width, vehicle width, and safety margin. Its programmatic `TrackCorridor` supports separate left/right width for *each source cell*, plus a required source description; the current desktop supplies equal uniform assumed widths because there is no surveyed corridor. The clearances must exceed $\text{vehicle_width}/2 + \text{margin}$ on both sides. These values define a center-of-car offset envelope, not an observed edge of pavement.

For a closed source course with length L_{source} , the planner removes the x/y endpoint mismatch linearly over source distance, interpolates x/y onto $N = \max(\text{ceil}(L_{\text{source}}/2 \text{ m}), 4 K, 32)$ periodic samples by default ($K=24$ controls), and applies wrapped Gaussian smoothing with a 0.8 m scale. It records closure error, maximum preparation shift, and processed path length difference. The processed zero-offset line is the **geometric baseline**; the ordinary source-curvature line is not a valid A/B baseline for a shifted x/y candidate. The planner refuses more than 5,000 samples, an open course, self-intersecting processed baseline, undefined tangent, degenerate cell, or unusable corridor.

At each processed sample, a central-difference tangent defines the unit left normal n_i . A periodic uniform cubic B-spline basis B_{ij} maps the K controls c_j to lateral offset $d_i = \sum_j B_{ij} c_j$; candidate sample positions are $p_i = p_{\text{base},i} + d_i n_i$. Because the basis is periodic, its offset and first derivatives meet at the lap seam. The algorithm computes candidate chord lengths $e11_i = |p_{i+1} - p_i|$, cumulative solver station $s_{i+1} = s_i + e11_i$, and signed vertex curvature from the adjacent chord vectors a and b :

```

kappa_vertex = 2 cross(a,b) / (|a| |b| |a+b|);
kappa_cell,i = (kappa_vertex,i + kappa_vertex,i+1) / 2;
J(c) = sum_i [kappa_cell,i^2 ell_i] / sum_i ell_i
      + lambda (sum_i ell_i / L_baseline - 1), lambda = 0.01 m^-2.

```

The discrete curvature is an approximation for the speed-envelope solver; it is recomputed for **both** geometric baseline and proposed path, never copied from the original source curvature. The first term of J has units m^{-2} ; the length-penalty coefficient therefore also has units m^{-2} , since the relative-length term is dimensionless. J favors lower average squared curvature with a weighted length penalty. It is **not** lap time, and it does not use car mass, power, grip, or aero while solving. SciPy SLSQP starts from all-zero offset, uses finite-difference objective derivatives, and is capped at 60 iterations by default. The numeric objective is multiplied by 10,000 for the optimizer's absolute tolerances; reported objective values remain unscaled. This is one local candidate, not a global racing-line optimum or a trained policy.

The linear optimizer constraints evaluate the spline at planner nodes and midpoints and at every original source-cell boundary and midpoint. Bounds at shared boundaries use the narrower adjacent width. A postcheck analytically finds the cubic offset extrema *within each source cell* against its piecewise constant left/right widths. The candidate must have violation no more than $1e-6$ m, forward local motion relative to the processed baseline, finite positive chord lengths, and no nonadjacent segment intersection. The latter check operates on the discretized polygon. Width certification is for the mathematical offset **relative to the processed reference and supplied cell widths**. It is not a surveyed cone clearance or a continuous swept-body collision check; the lateral normal field and true boundary geometry between stations are unknown.

`compare_lines_with_lap_model` copies the selected vehicle separately for the geometric centerline and every tested candidate so one lap's battery/vehicle state cannot contaminate another. All use the same torque fraction and unchanged `run_one_lap` physics. It tests the full-offset proposal first. If that path fails or is no faster than a completed baseline, it builds a half-offset path between the same geometric baseline and validated proposal. Convex offset scaling preserves the supplied lateral bounds; the half path independently rebuilds station/curvature and checks forward direction and nonadjacent segment crossings. There is one baseline call and at most two candidate calls, **three complete lap-model evaluations total**. The best completed candidate is selected only if both it and the baseline complete and it is strictly faster. If a candidate completes while its baseline fails, the desktop may show it for inspection but labels the comparison invalid and claims no gain. An optimizer stop without its success flag can still supply a feasible candidate that passes postchecks and improves the geometry objective. A path that reduces J can still lose the full car-dependent time comparison and is then rejected. This is a coarse car-dependent line-strength search, not minimum-time optimization over arbitrary paths. The lab's wind/road model does **not** feed this path planner or any lap call; the main lap remains flat, uniform and still-air.

This method is inspired by the boundary-aware, minimum-curvature stage in [TUM FTM's global racetrajectory optimization](https://github.com/TUMFTM/global_racetrajectory_optimization) and by the separation of path target, speed control, and edge checks in the [TORCS robot tutorial](https://torcs.sourceforge.net/api/robot_tutorial_chapter_4.html). The repository implements its own bounded spline/objective code, not copied project code. See `docs/ai_racer_design.md` for the focused research record and measured geometry mismatch.

9. What the main GUI displays

View/control	Source and meaning
Car profile / editable boxes	Fresh vehicle from <code>build_vehicle</code> or editable Prius-equivalent <code>VehicleSetup</code> ; seven car fields editable on Prius and saved copies
Driver request	Constant fraction, 0-100%, of motor torque-curve request before vehicle/path limits
Solver step	Maximum resampled source-curvature cell length in metres for default mode; AI mode independently builds a nominal 2 m geometric grid
Driving path	Centerline is default; experimental AI requires assumed corridor half-width, vehicle width, and safety margin
Run one lap	Worker executes one flying prescribed-path lap in default mode; AI mode plans one geometry candidate and tests full/optional half offset against its processed centerline
Compare car profiles	Two saved/built-in profiles run against the same course, step, and driver fraction; B-minus-A numbers are model differences
Six boxes	Lap time, peak speed, average speed (<code>course_length/time</code>), course distance, signed net pack-model energy, and peak <code>abs(lateral acceleration)/g</code>
AI result boxes	Geometric baseline and candidate model times, signed candidate-minus-baseline difference, selected path length; explanations state valid selection and assumed corridor
Trace selector	Speed, acceleration, drive/braking forces, inferred driven slip, or battery power from named telemetry channels, against time or distance
Course map	Shipped x/y reference, fixed top-down; mouse drag pans and wheel zooms; visible warning when map chords exceed assigned travel distance
Driver view tab	Timed, top-down path playback with fixed car marker, play/pause, scrub, rate, wheel zoom, and numeric time/distance/speed/lateral-g/map-heading boxes
Timed sessions · WIP tab	Scope text and disabled start button for the future ghost/controller/complete-session/replay-tolerance workflow
Source data / model notes	Inspect reviewed records, origins, model use, and caveats; inspection does not change the active vehicle
Dark mode	Reverses white/black Tk and Matplotlib surfaces; it does not change a simulation parameter

The GUI deliberately keeps numeric output boxes and simple monochrome traces. A solid line means A/current run and a dashed line means B in comparisons. Do not infer measurement uncertainty from line thickness, and do not interpret a visually overlapping trace as an identical force or energy history. For exact values and full channels use the saved JSON record.

9.1 Driver playback data flow

`ui/driver_view.py::DriverPlayback` requires four equal-length, finite, synchronized telemetry arrays: `vehicle.time_s`, `vehicle.distance_m`, `vehicle.speed_mps`, and `vehicle.lateral_acceleration_mps2`. It rejects nonincreasing time, decreasing distance, negative speed, and a station beyond the displayed path. Endurance telemetry begins after an accepted cell, so playback prepends a time-zero display frame at station zero when necessary. Its initial speed is recovered from the first cell's distance, exit speed, and elapsed time. Given a requested display time within a consistent cell, it interpolates speed and station with the same constant-acceleration relation as `Vehicle.update_state`; inconsistent imported cells use linear station interpolation instead. Lateral acceleration is interpolated from samples. It then interpolates x/y by station on the selected reference path and computes a displayed heading from nearby path points. The canvas transforms a short path segment into the car-fixed view; the triangle stays fixed while the map line rotates. The GUI uses wall-clock time only to advance the display cursor, with selectable 0.25× to 4× speed. Playback can be scrubbed and zoomed without rerunning physics.

For a default lap the displayed path is the shipped x/y course; for an AI lap it is the **selected processed** path that was timed. Neither is the integrated `Vehicle` x/y/heading state. In particular, the default track's x/y and curvature disagree numerically. The view is a useful time/station and speed preview but does **not** show steering error, tire path tracking, cone proximity, a true driver's camera, or interactive driving. The desktop switches to this tab and starts playback after a completed lap. The Timed sessions tab remains visibly unavailable until a closed-loop session engine, versioned controller contract, full input/state/environment capture, replay tolerance policy, and linked comparison report exist.

10. Four-wheel lab: state, control, tire, and motion equations

`src/lapsim/dynamics/planar.py` is a separate **ten-state** ordinary differential equation model. Its state is $(X, Y, \psi, u, v, r, \omega_{FL}, \omega_{FR}, \omega_{RL}, \omega_{RR})$: three planar pose values, three body motion values, and four wheel speeds. `PlanarControls` holds four steering angles, four signed drive torques, four nonnegative friction-brake torque requests, optional four normal-load overrides, and optional external body force/yaw moment. The desktop holds these controls constant throughout each A or B run; it sets equal left/right front steering and zero rear steering. An external controller can provide a different sequence to `run_planar_dynamics`. The GUI does not simulate a motor, inverter, pack, torque delivery latency, or controller feedback in this mode. A negative drive torque is a signed generating/reverse mechanical request, not an automatically constrained battery-regeneration command.

The lab's default synthetic vehicle uses $m=300$ kg, $I_z=160$ kg m², $a=b=0.8$ m, $t_f=t_r=1.2$ m, $R=0.2$ m, wheel $J=0.3$ kg m², $\mu=1.5$, $C_k=7000$ N/slip, and $C_a=8000$ N/rad. Its default 1 s maneuver starts at 12 m/s, front steer 1 degree, output step 0.01 s, and rear requests A (120,120) N m, B (110,130) N m; front torques are zero. These numbers are **test scenario inputs**, not a Prius or TREV calibration. They are visible and editable in the lab. `PlanarVehicleConfig` also holds gravity 9.80665 m/s², regularization floor 0.5 m/s, and maximum internal step 0.002 s unless code changes them.

For front axle distance a , rear distance b , and front/rear tracks t_f, t_r , wheel body positions are $(a, +t_f/2)$, $(a, -t_f/2)$, $(-b, +t_r/2)$, and $(-b, -t_r/2)$ in FL, FR, RL, RR order. With no override, static loads are $F_{z_FL}=F_{z_FR}=mg\ b/[2(a+b)]$ and $F_{z_RL}=F_{z_RR}=mg\ a/[2(a+b)]$. They do **not** change with acceleration, grade, bank, aero, or a grip patch. Passing `controls.normal_loads_n` is a prescribed load experiment; it is not suspension physics. A zero-load wheel produces zero tire force but its spin equation still evolves.

At wheel center (x_i, y_i) and steering angle δ_{θ_i} , the code computes contact velocity in the body frame and rotates it into that wheel's frame:

```
Vxb = u - r y_i;          Vyb = v + r x_i
Vxi = cos(delta_i) Vxb + sin(delta_i) Vyb
Vyi = -sin(delta_i) Vxb + cos(delta_i) Vyb
vref = max(abs(Vxi), v_floor)
slip_i = (R omega_i - Vxi) / vref
alpha_i = atan2(Vyi, vref)
Fx_raw_i = Ck slip_i;      Fy_raw_i = -Ca alpha_i
Fcap_i = mu * road_multiplier_i * Fz_i
scale_i = min(1, Fcap_i / hypot(Fx_raw_i, Fy_raw_i))
Fx_i = scale_i Fx_raw_i;   Fy_i = scale_i Fy_raw_i
```

When the raw force magnitude is zero, $scale_i=1$. The common scale is a circular **combined** force cap, so large requested longitudinal force reduces the achieved lateral force and vice versa. v_{floor} is 0.5 m/s in the lab's default synthetic car, preventing division by zero during launch or wheel lock. It changes the low-speed law; it is not a tire measurement. Reverse behavior is qualitative. The grip multiplier changes **only** the force cap; it does not change stiffness, contact load, wheel radius, or the slip formula. A measured tire lookup would need load, slip, camber, road condition, and temperature data with units and validity ranges.

The achieved wheel force is rotated back to body axes. The net yaw moment is $\sum(x_i F_{yb_i} - y_i F_{xb_i}) + M_{external}$, with positive yaw counterclockwise. A larger forward force at the right rear has a positive yaw contribution. The state derivatives are:

```
Xdot = u cos(psi) - v sin(psi); Ydot = u sin(psi) + v cos(psi)
psidot = r
udot = r v + (sum Fxb_i + Fx_external + Fx_aero)/m
vdot = -r u + (sum Fyb_i + Fy_external + Fy_aero)/m
rdot = (sum(x_i Fyb_i - y_i Fxb_i) + M_external)/Iz
J omega_dot_i = Tdrive_i + Tbrake_applied_i - R Fx_i.
```

Brake torque opposes actual wheel spin. At exactly zero spin it is capped to prevent a requested friction brake from creating wheel rotation in the opposite direction; integration also handles an incipient zero crossing. These brake rules are in the derivative/integrator, not a hydraulic pressure or ABS model. Unlike the main lap, the four wheel inertias are explicit energy and dynamic states, so they are **not** reflected into the body mass.

10.1 Power identity and integration

Every force evaluation computes $E = 0.5m(u^2+v^2) + 0.5I_z r^2 + 0.5J \sum(\omega_i^2)$. Mechanical drive and brake powers are $\sum(Tdrive_i \omega_i)$ and $\sum(Tbrake_applied_i \omega_i)$. Contact dissipation is $\sum[Fx_i(R \omega_i - V_{xi}) - Fy_i V_{yi}]$. External power is $Fx_external u + Fy_external v + M_external r + Fx_aero u + Fy_aero v$. The diagnostic is:

```
power_residual = E_dot - P_drive - P_brake
                + P_contact_dissipation - P_external.
```

It should be at roundoff for a force evaluation, including the added aero work. This identity checks sign and accounting consistency at an instant. It does not prove that an RK4 trajectory conserves energy exactly, that every requested torque is hardware-feasible, or that the tire parameters are real. The code records the diagnostic in `PlanarEvaluation` and shows it at the GUI cursor.

`run_planar_dynamics` uses classical fourth-order Runge-Kutta with held controls/environment over an output interval, splitting that interval into smaller internal steps. The cap is $\min(user_max_step, 0.8/\lambda)$, where $\lambda = (C_k/v_{floor})(R^2/J+4/m)$ approximates the fastest small-slip mode. The synthetic config's explicit maximum is 0.002 s; stiffness may force smaller substeps. The public run stores $N+1$ boundary times/states and N pre-control interval evaluations. There is no final-point evaluation because no control interval follows the final state; the GUI computes one only for display with the last held control. A single-step API raises on an invalid road query; the batch runner returns a flagged run. The output count and internal substep count have explicit upper bounds in code. The integration is deterministic for identical input values and code, apart from ordinary floating-point/runtime variation.

11. Report II upgrade: wind and road conditions

`src/lapsim/dynamics/conditions.py` defines immutable `PlanarEnvironment`, `PlanarRoad`, `RectangularGripPatch`, and optional `RoadDomain`. Wind is a **world-frame vector** (w_x, w_y). For heading ψ , the derivative first rotates it into the body frame, then subtracts it from ground velocity:

```
w_body = R(psi)^T w_world
U_body = (u,v) - w_body;      Vair = hypot(Ux,Uy)
F_aero_body = -0.5 rho CdA Vair U_body
P_aero = F_aero_body dot (u,v).
```

The final line uses **ground** velocity for the vehicle's mechanical power balance. Replacing it with $\mathbf{F} \cdot \mathbf{u}$ would account for a different energy transfer involving moving air. A headwind raises apparent speed and drag; a tailwind faster than the car can make the modeled aero force accelerate the car. C_dA is a single, isotropic, synthetic drag area placed at the center of gravity. A crosswind therefore produces a lateral component under this simple vector drag law, but the code does **not** claim a measured side-force coefficient, aero yaw moment, downforce, front/rear balance, CFD map, or ride-height coupling. `drag_area_m2=0` disables aero exactly and is the core API's backward-compatible default. The GUI starts with a clearly synthetic $C_dA=0.8 \text{ m}^2$, zero wind, and $\rho=1.225 \text{ kg/m}^3$; those values are study inputs, not team-car data.

For each wheel center, the code evaluates world position $(X+\cos(\psi)x_i-\sin(\psi)y_i, Y+\sin(\psi)x_i+\cos(\psi)y_i)$ and queries the road there. The base material is `reference_pavement` with multiplier 1. A finite rectangular patch can assign a different nonnegative multiplier and material identifier. Patches are world-fixed, inclusive of their boundaries, and searched in tuple order; the first matching patch owns an overlap. The GUI can turn on one assumed low-grip rectangle and shows its top-down outline. It passes the **same** environment object to A and B. The code makes no claim that the shipped endurance course has this road patch.

RoadDomain is a programmatic optional rectangle representing where road data are supported. If a wheel query leaves it, the road uses the base grip **as a diagnostic fallback** and sets `valid=False`; the result must not be ranked as a validated comparison. The batch run's `road_valid` flag aggregates all wheel force evaluations, including intermediate RK4 stages and the final state. `invalid_road_queries` counts invalid wheel-force queries, including repeated substep evaluations; it is neither elapsed time nor a count of unique places. The GUI reads the run-level flag, labels the result INVALID ROAD DOMAIN, and withholds the B-minus-A ranking. The GUI currently does not expose a domain editor; an API caller can supply one.

11.1 What Report II does and does not authorize

Report II is a design and verification source, not a command to copy its synthetic values into the team car. Its first defensible change is to keep the lap solver's established flat baseline while adding explicit air-relative wind and road contact queries to the independent-wheel lab. The report recommends a progression from scalar coefficients and flat road through measured six-component aero maps, surveyed elevation/bank/curvature, dynamic suspension/contact, and conservative thermal states. This implementation stops at the first environment layer because no reviewed team CFD map, ride-height schedule, tire/road friction map, elevation/bank survey, cell thermal characterization, or cooling test is available here.

The main distance-domain lap still uses ground speed as airspeed, a constant density, scalar drag/downforce, and flat uniform road. The new `PlanarEnvironment` is **not** passed into `run_one_lap`, `PathConstraintSolver`, or `Vehicle.update_state`. It would be incorrect to claim that a wind or low-grip value entered in the lab changes lap time. Connecting them safely requires a path-referenced world wind field, force/moment allocation without double counting, a contact-point road query compatible with the lap model, and new regression/validation cases. Sections 16-17 define the evidence gate for that work.

12. Four-wheel GUI inputs and outputs

`ui/dynamics_lab.py::ManeuverSettings` freezes a car config, initial speed, steering, duration, output step, four A torques, four B torques, equal-total requirement, and one shared environment. It validates finite values, speed 0-40 m/s, steer within 30 degrees, duration $(0, 10]$ s, output step 0.002-0.1 s, integral 1-2000 output steps, and per-wheel torque magnitude $\leq 500 \text{ N m}$. Equal total torque is on by default so an A/B yaw comparison isolates left/right allocation more clearly; it is a **sum of torque requests**, not a guarantee of equal electrical power or equal tire force. `run_maneuver_pair` gives A and B identical initial state, front steer, duration, output times, synthetic car, and environment. Initial wheel speeds are u_0/R .

The side panel exposes the car's mass, yaw inertia, geometry, wheel radius/inertia, tire μ_u , and two stiffnesses. Each entry is an explicit synthetic parameter. World X/Y wind, air density, drag area, base grip scale, and optional patch rectangle/grip scale appear in simple number boxes. A/B wheel torques appear in a four-row table. The plots show top-down paths, yaw-rate histories, four wheel-slip histories for the selected car, and a body-frame wheel diagram. The time slider changes only inspection. Numeric boxes show time, body forward/lateral speed, heading, yaw rate, yaw moment, peak tire use, instantaneous power residual, apparent airspeed, aero body X/Y forces, and whole-run road coverage. The coverage box says ASSUMED when no validity domain was supplied; otherwise it reflects the entire run, not only the cursor sample. The wheel

diagram shows each tire's slip, local longitudinal/lateral force, normal load, and road grip multiplier. Dark mode reverses black/white surfaces; it never changes physics.

The path is a computed open-loop **maneuver trajectory**, not the 989 m Formula SAE course. The main app's top-down course is a reference map. Neither view uses 3D animation, elevation, or moving-camera effects. A/B final yaw differences are reported only for domain-valid runs and must be interpreted as sensitivity of this synthetic model, not a control-system lap-time gain. The selected scenario and time cursor choose an evaluation already generated from the frozen settings; editing a text box does not retroactively change a displayed run until Compare A and B is pressed.

13. Profile, source-data, and model audit trail

The profile selector's name is a label; the **effective constructed vehicle** is the physics input. `profiles/registry.py` identifies built-ins and optional locally reviewed TREV source bundles. `profiles/adapters.py` maps allowlisted source values into a fresh `Vehicle`, records inherited defaults and unused fields, and returns a `ResolvedManifest`. The local editable garage in `ui/garage.py` is a small seven-field Prius-equivalent convenience layer. It should not be mistaken for a full calibration manager. A software lead can trace a lap number by reading, in order, the selected profile and overrides, the resolved manifest, the actual effective vehicle snapshot, the resampled track grid, solver settings, controller fraction, result status, and telemetry channels in its saved record.

Data and code are separated deliberately. The shipped course CSV and its metadata are historical inputs; `docs/model_parameters.md` and validation notes describe where baseline values came from; the two ENME408 PDFs are research/design evidence; local Downloads source bundles remain optional and are never silently invented. A source value is only promoted when adapter logic accepts its field and units. A missing 2026-27 parameter remains an inherited assumption or an explicit unknown. The code does not infer a two-motor topology, battery thermal parameters, CFD coefficients, tire temperature law, track bank, or road friction from the research prose.

The important reproducibility distinction is **model input versus measured evidence**. A JSON record's hashed model configuration and track grid are enough to inspect what that software run attempted. They are not a signed calibration certificate, a raw log archive, or proof that the course and car matched an event. The detailed run-record schema and replay gaps are in Section 14.

14. Saved records: exact schema, integrity, and replay limits

14.1 Main-lap record (`experiments/run_record.py`)

`LapRunSettings.from_track` freezes the **resampled solver grid**, not just a filename: `closed`, all distance/X/Y/curvature values, and a SHA-256 of canonical JSON for that geometry. It also saves course ID, length, cell count, requested maximum step, path-constraint settings, constant driver torque fraction, one-lap event settings, and selected profile identifiers. `capture_lap_run` combines these settings with the source-aware profile manifest, editable overrides, full effective `Vehicle` constructor-field snapshot and its hash, result status/reason/time/pack energy/final SOC, and synchronized post-cell telemetry. Each telemetry channel has values, unit when identified, origin category, sample count, and missing-sample status; a missing numeric sample is JSON `null`, not an undocumented zero. The record carries Python implementation/version, operating-system descriptors, and installed NumPy/SciPy/Matplotlib versions. `schema_version=2` embeds the grid; the loader accepts the older v1 format too.

For an AI-selected desktop run, `settings.path_planning` additionally identifies `periodic_cubic_minimum_curvature_slsqp_v2_bounded_strength`, the selected candidate/baseline status, a SHA-256 of the **source** geometry, requested centerline step, actual planner spacing/controls/smoothing/iteration cap/length penalty, the assumed left/right corridor source and dimensions, baseline result, each tested candidate offset strength/path length/time/error, best candidate and selected offset strength, maximum offset and constraint violation, source closure/preparation discrepancy, and measured planner/lap-comparison compute time. The usual embedded `settings.track.geometry` is the **selected processed solver path** and its hash, so a reviewer can inspect exactly what the selected lap used. A default run has no `path_planning` field. Optional planning metadata is validated as canonical finite JSON

at record construction, and the record's content ID covers it. The GUI currently saves only the **selected** lap record from an AI comparison: other full telemetry and generated paths are not embedded as a linked A/B package. Re-running requires the source file identified by the hash, equivalent environment/code, and the original selected vehicle inputs. Treat stored comparison times as reported results, not as a self-contained replay proof of both paths.

For AI-generated geometry, `path_planning.actual_maximum_cell_length_m` records the measured largest cell. The legacy solver field `requested_maximum_cell_length_m` receives that measured value for schema compatibility; `path_planning.planner_sample_spacing_m` separately records the nominal 2 m planner grid. The exact embedded cell distances, not either scalar, are the authoritative solver input.

The complete payload is serialized as stable, finite canonical JSON and SHA-256 is its content-derived `run_id`. `RunRecord.save` writes a temporary file and atomically replaces `%LOCALAPPDATA%\LapSim\runs\<run_id>.json`. `RunRecord.load` checks the ID and, for v2, validates embedded geometry, metadata, and geometry hash. Equal content under the same runtime/source identity gets the same name and overwrites that same content; the ID is not a per-attempt timestamp. A failed event result is saved when it reaches capture, but an exception before capture/save leaves no record. The GUI's A/B lap comparison creates two individual run files, not a third linked comparison manifest. Keep the two full IDs together when reviewing a comparison.

The embedded grid makes the solver input auditable even if the checkout course file later changes. It does not embed raw GNSS/IMU logs, original fusion scripts, or a signed track survey. The manifest records Git commit and dirty-worktree flag; it does not store the uncommitted patch, so a dirty run cannot be exactly reproduced from the commit alone. Dependency versions are selected key packages, not a complete lockfile or BLAS build. The effective snapshot is an inspection record, not a universal restore constructor. A content hash detects accidental mutation but does not authenticate who supplied the data.

14.2 Four-wheel A/B record (`experiments/dynamics_record.py`)

`capture_dynamics_comparison` takes the exact `PlanarVehicleConfig`, `PlanarEnvironment` (including wind, density, CdA, base road, domain, and ordered patches), initial state, every A/B control step, output step, and both `PlanarRun` objects. It checks both runs share the initial state/time grid, that equal-total torque holds at each step if requested, and then **replays both runs** under those frozen inputs before accepting their trajectories or RK-stage road status. The record stores **every** boundary time/state, every pre-step wheel/force/energy evaluation, per-sample body longitudinal/lateral force residuals, yaw-moment residuals, power residuals and their maxima, road-validity flags and invalid-query counts. It identifies the FL/FR/RL/RR wheel order, Python/platform/dependency versions, source commit/dirty flag, and synthetic evidence status.

`DynamicsComparisonRecord.load` checks its content ID, alignment/time grid, torque condition, equation residual summaries, and deterministic replay against the saved input. Both A and B are in one linked file at `%LOCALAPPDATA%\LapSim\dynamics_runs\<run_id>.json`. The UI saves it before displaying a completed comparison and shows the shortened ID.

The lab record supports inspection and programmatic replay from exact inputs, but there is no one-click replay button yet. Its power/force residuals are evaluated from the saved numerical derivatives and forces; they do not compare a simulated maneuver with instrumented car measurements. The road-valid flag reports that the **solver found** every queried contact inside the declared domain; capture and load rerun the integrator to check this, including RK4 stages. The file still stores only their aggregate invalid-query count, not every intermediate contact coordinate. If no domain exists, coverage is an unbounded synthetic assumption, which the GUI calls `ASSUMED`; it is not a claim of road survey coverage. A/B record capture does not include the current uncommitted source patch. Because load uses the installed model code for replay, a future change to physics or integration may reject a historically valid record until an explicit schema/model-version migration or archived-code replay is provided.

14.3 Practical record inspection

Read the JSON in this order: `schema_version` and `simulation_mode`; `runtime` and source identity; `configuration/inputs`; `settings` and solver grid or time step; `result/runs`; `validity`; then `telemetry/evaluations`. A run is usable for a model comparison only if its event succeeded, its selected inputs match the intended comparison, its declared validity checks pass, and its assumed parameters are appropriate for the engineering question. Never rank a failed lap or a road-out-of-domain maneuver by its partial elapsed time or final yaw alone. Preserve the full JSON alongside any plot shared with the team.

15. How Python modules and the desktop interact

Responsibility	Main source	Contract / important side effect
Desktop entry and main widgets	<code>src/lapsim/ui/__main__.py</code> , <code>ui/app.py</code>	Tk event loop; validates form inputs; dispatches worker; renders results on GUI thread
Four-wheel widgets	<code>ui/dynamics_lab.py</code>	Freezes <code>ManeuverSettings</code> , runs matched A/B in worker, saves one record, draws top-down/force views
Shipped course and gridding	<code>ui/simulation.py</code> , <code>tracks/spatial.py</code>	Reads source CSV and creates path cells; step size affects solver input
Optional path proposal and A/B timing	<code>optimization/racing_line.py</code>	Builds processed geometric baseline and one bounded candidate; checks corridor/geometry and calls the unchanged lap model at most twice
Car selection and local saved profiles	<code>profiles/registry.py</code> , <code>profiles/adaptor.py</code> , <code>ui/garage.py</code>	Constructs fresh <code>Vehicle</code> and provenance; seven editable garage fields
Spatial speed constraints	<code>solvers/path_constraints.py</code>	Computes local tire-limited and cyclic braking entry ceilings
One-lap event	<code>events/endurance.py</code>	Starts flying lap, applies driver/automatic brake actions, calls cell update, records accepted telemetry
Main vehicle assembly	<code>vehicle_model/vehicle.py</code>	Couples aero, suspension, tire, drivetrain, brake, battery and commits state after solving a cell
Four-wheel derivative and integrator	<code>dynamics/planar.py</code>	Pure force/derivative evaluation; bounded RK4 time integration; no lap score
Wind/road scenario	<code>dynamics/conditions.py</code>	Deterministic world wind and world-fixed contact-patch query; no update side effect
Main-lap evidence	<code>experiments/run_record.py</code>	Content-hashed v2 record with exact solver grid and effective vehicle snapshot
Driver view interpolation	<code>ui/driver_view.py</code>	Maps completed time/station telemetry to selected reference x/y without changing physics
Four-wheel evidence	<code>experiments/dynamics_record.py</code>	Linked A/B content-hashed input/trajectory/force record

An implementation change should begin with the **contract** between these modules. For example, adding measured wind to the lap solver is more than another GUI box: the track needs an orientation/world frame, the path solver must use air-relative forces at its speed-ceiling probes, `Vehicle.update_state` must use the same force without double counting, the record must store the wind field and its frame, and analytical baseline/headwind/crosswind checks must pass. Likewise, adding dynamic ride height requires new continuous states/contact geometry in the time-domain model or a defined equilibrium map in the distance-domain one; a plotted height slider alone would have no physical effect.

15.1 What to change for a new car profile

Create a reviewed parameter table with unit, source, date, vehicle configuration, operating range, and uncertainty. Add an allowlisted adapter mapping, then inspect the resolved manifest and effective `vehicle` snapshot. Confirm motor count/topology, wheel radii, torque/RPM/power envelope, cell/pack limit behavior, mass/CG/inertia, aero convention, tire load/slip data, braking/regen boundaries, and track condition separately. Do not fill missing fields with a plausible number without labeling it an assumption. The garage's seven boxes are useful for an initial sensitivity run, not enough to assert a 2026-27 calibration. Re-run the baseline unit/equation checks and held-out data comparison after any physical parameter update.

15.2 What to change for a new track

Supply a source-controlled or recorded course with monotone distance, x/y, signed curvature, closure decision, units, and derivation metadata. `resample_track` can create solver cells, but a new course should be checked for geometric closure, map/curvature agreement, and cone/path width. A legitimate Report II road extension additionally needs elevation, grade, bank, crest curvature, surface material/condition, and validity coverage referenced to a shared world frame. Compute gravity along and normal to the road consistently: grade changes required climbing work and normal force; bank changes lateral tire demand; a crest can unload contact. Those are **not** currently in the lap model. A local wetness or grip multiplier without a surface measurement should stay a sensitivity scenario.

15.3 Numerical and code-review guardrails

Keep derivative evaluations pure during RK4 and commit thermal, charge, wear, or weather-state evolution only after an accepted step. Keep one definition for each force and sign; aero loads must enter both force balance and power/accounting exactly once. Define whether a coefficient maps body velocity, air-relative velocity, or wheel-contact velocity. If adding pitch/yaw aero moments, verify moment reference points and equilibrium; if adding rotating states, remove any equivalent-mass term representing the same inertia. Preserve dimensional units at API boundaries. Version every new record field/schema and add a migration or explicit rejection path. The deterministic baseline and limiting-case checks in Section 16 are the minimum regression gate.

16. Checks performed, interpretation, and known discrepancies

The repository tests exercise constructors, component limits, path constraints, endurance events, desktop settings, record integrity, and the four-wheel equations. The environment checks cover backward-compatible zero-drag/still-air behavior, headwind/tailwind/crosswind signs and power, rotated world wind, one-sided grip and yaw, road queries after heading rotation, malformed inputs, deterministic repeated runs, and road-domain violations that occur **only inside RK4 stages**. The four-wheel lab checks that A/B share initial conditions and the same wind/patch, and that its worker saves a linked record. The record checks cover content hashes, geometry/input alignment, schema compatibility, tamper rejection, and residual summaries. The aero/suspension guards reject unsupported loss of normal reaction. On 6 October 2026, the Section 2 command completed with **242 tests passing and 32 subtests passing** in 45.76 s with `OPENBLAS_NUM_THREADS=1` and `OMP_NUM_THREADS=1`. The exact command, result, and Git commit should be logged together for a later design review.

Focused planner regression checks exercise explicit corridor requirements, x/y-derived rather than copied curvature, exact chord/station reconstruction, periodic geometry, deterministic bounded iteration, variable widths, narrow 0.5 m source cells, self-crossing rejection, conditional half-offset trials, failure handling, and candidate selection with the full lap model. The narrow-cell QA case permits only about ± 0.01 m offset; its reported candidate offset was about -0.00916 m with zero computed bound violation. A synthetic rounded rectangle with the built-in Prius at fixed 0.8 torque fraction gave a 16.77 s processed baseline and 14.23 s candidate. Separate `SpatialTrack.geometry_audit()` tests catch map chords longer than their station cells and accept shorter geometric chords around curves. Driver-view checks include constant-acceleration interpolation, inconsistent legacy fallback, and closed-course seam wrapping. These tests establish model/software behavior, not surveyed Formula SAE feasibility.

An illustrative shipped-course case used the built-in Prius at torque fraction 1.0 with a **user-assumed** ± 2 m half-width, 1.78308 m car width, and 0.3 m safety margin. The x/y-derived geometric baseline had about 1,011.04 m path length and completed in **87.168 s**; the full-offset candidate completed in **86.984 s**, so the selection gate chose it without a third lap call. At assumed ± 3 m, the full-offset candidate took **87.189 s** and lost, whereas the conditional half-offset trial took **86.986 s** and won. An ordinary 2 m source-curvature Prius run completed in about **78.778 s**, but its 989 m station path is not the same geometry and must **not** be subtracted from an AI time. At an assumed ± 2 m corridor, 1.8 m width and 0.2 m margin, the TREV5 working-geometry scenario completed the geometric baseline in **61.800 s**, full candidate in **61.838 s**, and half candidate in **61.806 s**; the baseline remained selected. The pressure-limit guard described in Section 8.2 was needed for these paths to complete. The team's width, car limits, and track conditions are not established by these synthetic scenarios.

These are software and equation checks. For an actual car model, additional independent evidence is needed: measured acceleration/coast/braking on specified surface, wheel torque/RPM and pack voltage/current, synchronized yaw/IMU/GNSS, tire force/slip/load and temperature, measured aero balance or CFD with a declared coordinate convention and validation, and surveyed course/ambient conditions. Fit one subset of runs, then test on held-out days, speeds, steering maneuvers, and tracks. Report time error, speed/yaw residuals, energy error, constraint violations, and parameter uncertainty separately. A single matched lap time can hide compensating wrong drag, grip, power, and brake models.

Existing historical material is useful but limited. `docs/model_parameters.md` states that rolling radius, motor efficiency, and chain efficiency were fitted to the same endurance data used for a historical comparison, so that agreement is **in-sample**. `docs/first_lap_soc_validation.md` records 39.07 s simulated versus 69.49 s measured for one first-lap comparison and 13.28 m/s speed RMSE; some discussion there predates later brake logic and should be rechecked before quoting it as current behavior. `docs/real_accel_validation_report.md` documents a rear-wheel-derived acceleration target and explains why it is not a clean independent gate. `docs/battery_rc_validation.md` gives a limited chronological voltage holdout for one historical pack log (about 2.669 V RMSE over its final 35%). None of these constitute a complete 2026-27 Formula SAE vehicle validation, a tire thermal calibration, or a Report II CFD proof.

Important current bounds and gaps:

Boundary	Consequence for interpretation
Main lap follows prescribed curvature and no cone/width rule	A fast time is not proof of a feasible driven line
AI offset check uses a declared synthetic corridor relative to processed x/y	It verifies a mathematical offset under supplied widths, not real cone clearance or swept-body collision
AI objective is local mean-curvature optimization plus bounded line-strength trials	A candidate can be slower for a car; up to three full-model laps choose baseline/full/half offset without proving global minimum time
Source x/y and curvature disagree	AI geometric baseline and ordinary source-curvature lap describe different tracks and must not be compared as a racing-line gain
Driver view maps solved station onto reference x/y	The marker is not the dynamically integrated vehicle pose or an interactive driver
Main lap uses flat, uniform, still air	Lab wind/road changes do not change lap time
Main tire is capacity-based with inferred slip	No independent wheel transient or temperature prediction in a lap
Lab uses static Fz and synthetic linear/circular tire	It demonstrates yaw/force coupling, not a calibrated car
Lab aero is vector drag at CG, no downforce/moments	Crosswind sensitivity is qualitative without measured coefficients
Lab road patch changes grip cap only	No roughness, rolling resistance, puddling, hydroplaning, or surface heat

Boundary	Consequence for interpretation
No thermal states in either current route	Continuous torque, brake temperature, tire fade, and pack derating cannot be predicted
Main event can commit a rejected final cell before its post-check	A failed result time/state may be ahead of accepted telemetry
No packaged EXE or full dependency lock	Deployment is from a configured source checkout and venv

17. Prioritized engineering work and acceptance evidence

- 1. Freeze the 2026-27 car evidence.** Collect a versioned, reviewed configuration: mass/CG/inertias, wheelbase/tracks/radii, drive topology and torque limits, pack/thermal data, brake/regen limits, tire map, and aero coordinate convention. Keep source, units, validity range, and uncertainty per field. Promote only verified values into adapters; keep unknowns explicit.
- 2. Establish held-out baseline validation.** Instrument a known dry, flat maneuver and course section. Align times and frames; compare speed, yaw, wheel speeds/torques, pack voltage/current, and brake response with error and uncertainty. Require zero-weather/constant-surface regression before each environmental change.
- 3. Survey the course before ranking racing lines.** Measure both cone boundaries in one world frame with x/y, station, closure, and uncertainty. Reconcile 989 m station/curvature with roughly 1,012 m x/y using retained raw data and versioned processing. Check that station and curvature describe the same path, then test the full swept vehicle envelope for boundary collisions. Until then, use a labeled synthetic corridor only for software checks.
- 4. Extend wind and aero only with reviewed maps.** A first lap wind extension needs a world-referenced course and measured/declared wind. A full six-component CFD or wind-tunnel map needs force/moment reference, yaw sign, ride height/roll/pitch axes, interpolation bounds, and crosswind benchmarks. Avoid applying both scalar drag/downforce and the same map forces.
- 5. Add surveyed road geometry and contact states.** Elevation/grade, bank, crest, material, and patch coverage must be sampled at actual wheel contacts. Check analytically that still-flat results recover baseline; constant grade gives longitudinal gravity force $mg \sin(\theta)$, elevation work $mg \Delta h$, and normal component $mg \cos(\theta)$; symmetric bank changes lateral demand with correct sign; crest unloading cannot produce negative contact loads. Track width and cone feasibility require a path model, not a display outline.
- 6. Add conservative thermal states only with calorimetry and cooling evidence.** Tire/brake/motor/inverter/pack temperatures need explicit heat capacities, generated heat, conductive/convective/radiative flows, ambient or coolant boundaries, and state-dependent limits. Internal heat exchanges must cancel in the whole-system balance. Slip and brake work must be assigned once, without double counting pack discharge. A derivative call must not change temperature or SOC.
- 7. Build timed sessions only after tracking and replay contracts exist.** A versioned vehicle and controller must record controls, states, environment, validity, and timing in one frame. A ghost is only a time/station comparison until both tracked poses are verified. Define deterministic replay inputs and channel tolerances; link both run IDs, source hashes, and software versions in a comparison manifest. Then make the disabled tab a testable workflow.
- 8. Harden distribution and replay.** Package course/profile data or document the source-only requirement. Pin dependencies, add replay for both record types, capture dirty patches or require clean code for comparison evidence, and save a linked lap A/B manifest. Document schema migrations and machine-independent tolerances.

Report II's examples and reference checks provide **test design**, not acceptance evidence for this checkout. Each stage above should have an equation/units check, limiting-case regression, synthetic sensitivity with monotonic expectations where applicable, and an independently measured holdout before it is described as predictive. Until then, label displayed results as model estimates and keep the exact record with any engineering decision.